서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험II

실험 전 보고서

LAB7: STM32_3 SPI 통신 및 MCU·FPGA 연동

학과	전자전기컴퓨터공학부
이름/학번	김영서 / 2025440031
작성일	2026년 11월 2일
대상	SPI 과제8~12 · MCU/FPGA 송수신
통신 조건	Mode 0 · 8비트 · MSB 우선 · SCK 1 MHz

이하

검증 환경	NUCLEO-G474RE / HBE-Combo II-DLD / 오실로스코프

1. 실험 목적 및 사전 준비 범위

1.1 실험 목적

이번 LAB7의 목적은 SPI의 동기식 직렬 통신 원리를 이해하고, STM32와 FPGA가 각각 마스터와 슬레이브 역할을 수행하도록 구성하는 것이다. MCU 송신 파형 확인, MCU 간 송수신, MCU→FPGA 수신, FPGA→MCU 송신, FPGA 간 송수신을 순차적으로 검증한다.

1.2 공통 조건

항목	조건
SPI 형식	Mode 0 · 8비트 · MSB first
클록	대기 LOW · SCK 1 MHz 이하
선택 신호	CS/NSS LOW 동안 통신
데이터	한 방향 MOSI 사용 · MISO 미연결
전압/접지	3.3 V 논리 · 공통 GND
MCU 확인	VCP UART 9600 8N1
실험일	2026년 11월 2일

표 1. LAB7 공통 실험 조건

2. SPI 동작 원리

2.1 Mode 0 타이밍

Mode 0은 CPOL=0, CPHA=0이다. 통신하지 않을 때 SCK는 LOW이며 수신 측은 상승 에지에서 MOSI를 읽는다. 송신 측은 첫 상승 에지 전에 MSB를 준비하고 이후 하강 에지마다 다음 비트를 출력한다. 상승 에지 8회가 한 바이트를 구성한다.

구간	SCK	MOSI/수신 동작
대기	LOW	CS=HIGH, 선택 해제
전송 시작	첫 상승 전	MSB(bit7) 준비
수신	상승 에지	현재 MOSI 비트 샘플
다음 비트	하강 에지	bit6→bit0 순으로 갱신
전송 종료	8번째 상승 후	CS=HIGH, 1바이트 완료

표 2. SPI Mode 0의 에지별 동작

2.2 예상 비트열

송신값	MSB→LSB
0x55	0 1 0 1 0 1 0 1
0xA5	1 0 1 0 0 1 0 1
0x00	0 0 0 0 0 0 0 0
0xFF	1 1 1 1 1 1 1 1

표 3. 오실로스코프에서 확인할 예상 비트열

SCK의 주기 T를 측정하면 주파수는 f=1/T로 계산한다. MCU 기본 SCK는 약 781.25 kHz이므로 주기는 약 1.28 µs로 예상한다.

3. 하드웨어 연결과 역할

3.1 MCU 마스터→MCU 슬레이브

신호	마스터(송신)	슬레이브(수신)
SCK	PB3	PB3
MOSI	PA7	PA7
CS/NSS	PB6	PA4
GND	GND	GND

표 4. MCU-MCU SPI 연결표

MCU Master	→	SCK·MOSI·CS	→	MCU Slave	→	VCP UART

그림 1. MCU→MCU 수신 확인 경로

역할을 교대할 때 프로그램뿐 아니라 CS 연결도 새 마스터의 PB6에서 새 슬레이브의 PA4로 바꾼다. 수신 MCU를 먼저 실행해 대기시킨 뒤 송신 MCU를 실행한다.

3.2 MCU-FPGA 공통 연결

- SCK, MOSI, CS, GND 네 선만 연결한다.
- MCU가 수신기이면 FPGA CS를 MCU PA4에 연결한다.
- MCU 핀에는 XDC를 사용하지 않으며 FPGA 측 핀만 XDC에 지정한다.
- 전원 인가 전에 3.3 V 논리와 공통 GND를 확인한다.

4. 과제8 · MCU 송신 파형 확인

4.1 송신 프로그램

```
uint8_t tx = 0x55;
HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
                  SPI_CS_ece2_Pin, GPIO_PIN_RESET);
HAL_StatusTypeDef result =
    HAL_SPI_Transmit(&hspi1, &tx, 1U, 100U);
HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
                  SPI_CS_ece2_Pin, GPIO_PIN_SET);
printf("TX=%02X, result=%d\r\n", tx, result);
```

그림 2. MCU SPI 1바이트 송신 핵심 코드

```c
#include "ece2_474.h"
#include "main.h"
#include <stdint.h>
#include <stdio.h>

void ece2_474_init(void)
{
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port, SPI_CS_ece2_Pin, GPIO_PIN_SET);
}

void polling_routine(void)
{
    static uint32_t last_ms = 0U;
    static const uint8_t patterns[] = {0x55U, 0xA5U, 0x00U, 0xFFU};
    static uint32_t index = 0U;
    uint32_t now_ms = HAL_GetTick();
    if ((now_ms - last_ms) < 1000U) return;
    last_ms = now_ms;

    uint8_t tx = patterns[index];
    index = (index + 1U) % (sizeof(patterns) / sizeof(patterns[0]));
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port, SPI_CS_ece2_Pin, GPIO_PIN_RESET);
    HAL_StatusTypeDef result = HAL_SPI_Transmit(&hspi1, &tx, 1U, 100U);
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port, SPI_CS_ece2_Pin, GPIO_PIN_SET);
    printf("TASK8 TX=%02X result=%d\r\n", (unsigned int)tx, (int)result);
}
```

그림 7. 과제8 STM32 SPI 송신 코드

```
Executing task in folder FPGA_LAB7: C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\Users\young\Documents\fpga_lab7/stm32/tools/activate_and_build.ps1 -Task task8

Cleaning... 0 files.
ninja: Entering directory `C:\Users\young\Documents\ece2_474\build\Debug'
[31/31] Linking C executable ece2_474.elf
Memory region         Used Size  Region Size  %age Used
           RAM:        2648 B       128 KB      2.02%
         FLASH:       20800 B       512 KB      5.02%
ACTIVE task8; backup=C:\Users\young\Documents\ece2_474\lab7_backup\20261102-114439_before_task8
```

그림 8. 과제8 STM32 빌드 성공 결과

1초마다 0x55를 전송하고 0xA5, 0x00, 0xFF로 값을 바꾸어 비교한다. result=0은 HAL 함수가 정상 종료했다는 뜻이며 실제 수신 성공은 상대 장치 출력에서 별도로 판단한다.

4.2 오실로스코프 확인 계획

관찰 항목	판정 기준
CS	LOW 구간에서만 8비트 전송
SCK	대기 LOW, 상승 에지 8회
MOSI	상승 에지 기준 예상 비트열 일치
주파수	측정값 1 MHz 이하

표 5. 과제8 파형 판정 기준

5. 과제9 · MCU→MCU 수신 확인

5.1 수신 측 설정

수신 MCU는 SPI1을 SLAVE, NSS를 HARD INPUT으로 다시 초기화하고 PA4를 SPI1 NSS(AF5) 입력으로 설정한다. HAL_SPI_Receive()로 한 바이트를 최대 2초 기다린 후 VCP UART로 RX 값을 출력한다.

```
uint8_t rx = 0;
HAL_StatusTypeDef result =
    HAL_SPI_Receive(&hspi1, &rx,
1U, 2000U);
if (result == HAL_OK)
    printf("RX=%02X\r\n", rx);
else if (result == HAL_TIMEOUT)
    printf("RX timeout: check sender,
SCK, MOSI, CS and GND\r\n");
```

그림 3. MCU SPI 수신 및 VCP UART 출력 코드

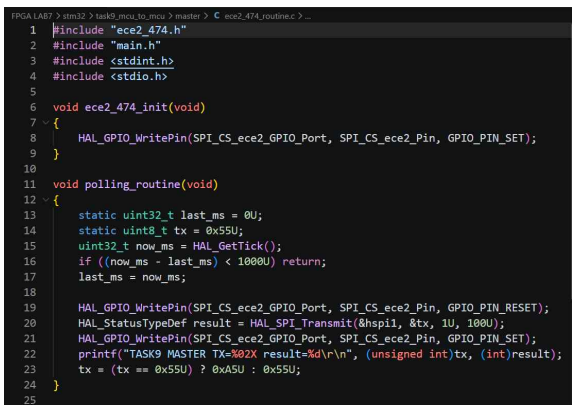


그림 9. 과제9 MCU 마스터 송신 코드

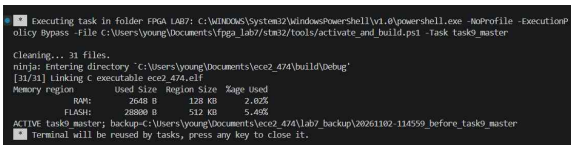


그림 10. 과제9 MCU 마스터 빌드 성공 결과

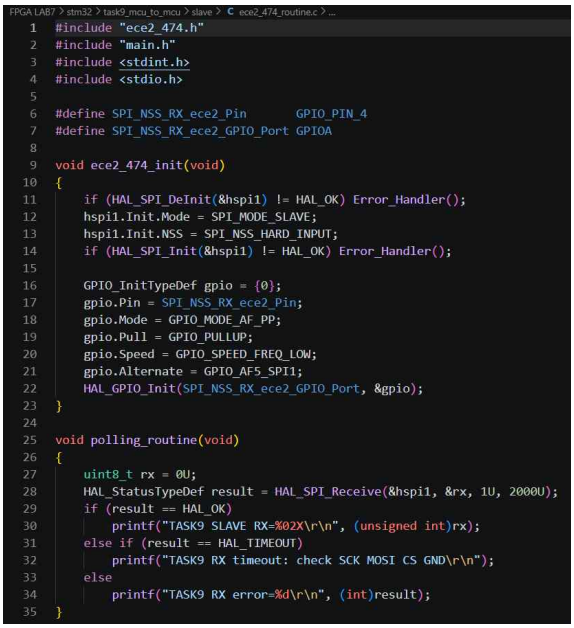


그림 11. 과제9 MCU 슬레이브 수신 코드

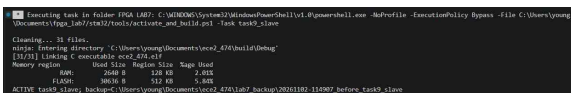


그림 12. 과제9 MCU 슬레이브 빌드 성공 결과

5.2 검증 순서

- 지정 조와 SCK·MOSI·CS·GND를 연결한다.

- 슬레이브를 먼저 실행하고 VCP Serial Monitor를 연다.
- 마스터를 실행해 TX와 RX 값이 동일한지 확인한다.
- 역할과 CS 배선을 교대해 양쪽 보드에서 송신·수신을 모두 검증한다.

송신기가 실행되지 않았을 때의 RX timeout은 정상적인 대기 결과다. 반복 송신 중 값 누락이나 비트 반전이 없어야 성공으로 판단한다.

6. 과제10 · MCU→FPGA 수신 회로

6.1 FPGA 수신 구조

입력동기화	→	SCK상승검출	→	8-bit Shift	→	수신완료	→	UART출력

그림 4. FPGA SPI 슬레이브 수신 데이터 경로

외부 SCK·CS·MOSI는 FPGA 시스템 클록과 비동기이므로 2단 플립플롭으로 동기화한다. 동기화된 SCK의 이전값과 현재값을 비교하여 상승 에지를 검출하고, CS가 LOW일 때만 시프트 레지스터와 비트 카운터를 갱신한다.

6.2 상태와 예외 처리

조건	동작
CS=HIGH	카운터 초기화, 다음 프레임 대기
CS=LOW & SCK↑	shift={shift[6:0], MOSI}
8번째 SCK↑	rx_data 저장, rx_valid 1클록
CS 조기 해제	불완전 바이트 폐기

표 6. FPGA SPI 수신기 제어 조건

6.3 사전 테스트벤치

```c
#include "ece2_474.h"
#include "main.h"
#include <stdint.h>
#include <stdio.h>

void ece2_474_init(void)
{
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port, SPI_CS_ece2_Pin, GPIO_PIN_SET);
}

void polling_routine(void)
{
    static uint32_t last_ms = 0U;
    static uint8_t tx = 0x55U;
    uint32_t now_ms = HAL_GetTick();
    if ((now_ms - last_ms) < 1000U) return;
    last_ms = now_ms;

    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port, SPI_CS_ece2_Pin, GPIO_PIN_RESET);
    HAL_StatusTypeDef result = HAL_SPI_Transmit(&hspi1, &tx, 1U, 100U);
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port, SPI_CS_ece2_Pin, GPIO_PIN_SET);
    printf("TASK10 MCU->FPGA TX=%02X result=%d\r\n", (unsigned int)tx, (int)result);
    tx = (tx == 0x55U) ? 0xA5U : 0x55U;
}
```

그림 13. 과제10 MCU→FPGA 송신 코드

```
Executing task in folder FPGA_LAB7: C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\Users\young
\Documents\fpga_lab7/stm32/tools/activate_and_build.ps1 -Task task10

Cleaning... 31 files.
ninja: Entering directory `C:\Users\young\Documents\ece2_474\build\Debug`
[31/31] Linking C executable ece2_474.elf
Memory region         Used Size  Region Size  %age Used
            RAM:       2648 B       128 KB      2.02%
          FLASH:      20804 B       512 KB      3.99%
ACTIVE task10, backup=C:\Users\young\Documents\ece2_474\lab7_backup\20261102-115002_before_task10
```

그림 14. 과제10 STM32 빌드 성공 결과

```
`timescale 1ns/1ps

module spi_slave_rx(
    input  wire       clk_50mhz,
    input  wire       rst_p,
    input  wire       spi_sck,
    input  wire       spi_cs_n,
    input  wire       spi_mosi,
    output reg  [7:0] rx_data,
    output reg        rx_valid
);
    reg [1:0] sck_sync = 2'b00;
    reg [1:0] cs_sync = 2'b11;
    reg [1:0] mosi_sync = 2'b00;
    reg       sck_prev = 1'b0;
    reg [7:0] shift = 8'h00;
    reg [2:0] bit_count = 3'd0;

    wire sck_rise = sck_sync[1] & ~sck_prev;

    always @(posedge clk_50mhz) begin
        if (rst_p) begin
            sck_sync <= 2'b00; cs_sync <= 2'b11; mosi_sync <= 2'b00;
            sck_prev <= 1'b0; shift <= 8'h00; bit_count <= 3'd0;
            rx_data <= 8'h00; rx_valid <= 1'b0;
        end else begin
            sck_sync <= {sck_sync[0], spi_sck};
            cs_sync <= {cs_sync[0], spi_cs_n};
            mosi_sync <= {mosi_sync[0], spi_mosi};
            sck_prev <= sck_sync[1];
            rx_valid <= 1'b0;
            if (cs_sync[1]) begin
                bit_count <= 3'd0;
            end else if (sck_rise) begin
                shift <= {shift[6:0], mosi_sync[1]};
                if (bit_count == 3'd7) begin
                    rx_data <= {shift[6:0], mosi_sync[1]};
                    rx_valid <= 1'b1;
                    bit_count <= 3'd0;
                end else begin
                    bit_count <= bit_count + 3'd1;
                end
            end
```

그림 15. 과제10 FPGA SPI 슬레이브 수신 코드(1)



```
            end
        end
    end
endmodule

module uart_tx #(
    parameter integer CLK_HZ = 50_000_000,
    parameter integer BAUD = 9600
)(
    input wire clk, input wire rst_p, input wire start,
    input wire [7:0] data, output reg txd, output reg busy
);
    localparam integer DIV = CLK_HZ / BAUD;
    reg [31:0] count = 0;
    reg [3:0] bit_index = 0;
    reg [9:0] frame = 10'h3ff;
    always @(posedge clk) begin
        if (rst_p) begin txd<=1'b1; busy<=1'b0; count<=0; bit_index<=0; frame<=10'h3ff; end
        else if (!busy && start) begin frame<={1'b1,data,1'b0}; busy<=1'b1; count<=0; bit_index<=0; txd<=1'b0; end
        else if (busy && count == DIV-1) begin
            count<=0;
            if (bit_index == 9) begin busy<=1'b0; txd<=1'b1; end
            else begin bit_index<=bit_index+1'b1; txd<=frame[bit_index+1'b1]; end
        end else if (busy) count<=count+1'b1;
    end
endmodule

module spi_slave_top(
    input wire clk_50mhz, input wire rst_p,
    input wire spi_sck, input wire spi_cs_n, input wire spi_mosi,
    output wire uart_txd, output wire [7:0] led_out
);
    wire [7:0] rx_data; wire rx_valid; wire uart_busy;
    spi_slave_rx u_rx(clk_50mhz,rst_p,spi_sck,spi_cs_n,spi_mosi,rx_data,rx_valid);
    uart_tx u_uart(clk_50mhz,rst_p,rx_valid && !uart_busy,rx_data,uart_txd,uart_busy);
    assign led_out = rx_data;
endmodule
```

그림 16. 과제10 FPGA SPI 슬레이브 및 UART 상위 코드(2)



```
Executing task in folder FPGA LAB7: C:/Users/young/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/python.exe C:\Users\young\Documents\fpga_lab7/circuits/tools/fpga_lab.py simulate

RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -s tb -o C:\Users\young\Documents\fpga_lab7\circuits\spi_slave\build\sim\sim.vvp C:\Users\young\Documents\fpga_lab7\circuits\spi_slave\src\spi_slave.v C:\Users\young\Documents\fpga_lab7\circuits\spi_slave\sim\tb_design.v
VCD info: dumpfile wave.vcd opened for output.
PASS SPI SLAVE: 55 A5 00 FF, CS ignore, one valid per byte
C:\Users\young\Documents\fpga_lab7\circuits\spi_slave\sim\tb_design.v:21: $finish called at 23500000 (1ps)
WAVE C:\Users\young\Documents\fpga_lab7\circuits\spi_slave\build\sim\wave.vcd
```

그림 17. 과제10 SPI 슬레이브 자기검사 PASS 결과

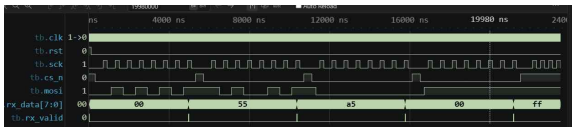


그림 18. 과제10 55·A5·00·FF 수신 파형

- 0x55, 0xA5, 0x00, 0xFF의 MSB-first 수신
- CS 해제 후 다음 바이트가 독립적으로 수신되는지 확인
- CS가 HIGH일 때 SCK 변화가 무시되는지 확인
- 수신 완료 펄스가 바이트당 한 번만 발생하는지 확인

7. 과제11 · FPGA→MCU 송신 회로

7.1 FPGA 마스터 구조

1초 Tick	→	CS LOW	→	SCK 분주	→	MOSI Shift	→	CS HIGH

그림 5. FPGA SPI 마스터 송신 순서

50 MHz 시스템 클록을 분주하여 1 MHz 이하 SCK를 만들고 1초마다 한 바이트를 전송한다. Mode 0 규칙에 따라 CS를 LOW로 내린 뒤 첫 상승 에지 전에 bit7을 MOSI에 출력한다. 상승 에지에서 슬레이브가 읽도록 유지하고, 하강 에지에서 다음 비트를 준비한다.

상태	출력/처리
IDLE	CS=1, SCK=0, 1초 Tick 대기
START	CS=0, MOSI=tx[7] 준비
TRANSFER	SCK 8주기, 하강마다 다음 비트
DONE	SCK=0, CS=1, 다음 전송 대기

표 7. FPGA SPI 송신 FSM

7.2 MCU 수신 확인

```c
#include "ece2_474.h"
#include "main.h"
#include <stdint.h>
#include <stdio.h>

#define SPI_NSS_RX_ece2_Pin       GPIO_PIN_4
#define SPI_NSS_RX_ece2_GPIO_Port GPIOA

void ece2_474_init(void)
{
    if (HAL_SPI_DeInit(&hspi1) != HAL_OK) Error_Handler();
    hspi1.Init.Mode = SPI_MODE_SLAVE;
    hspi1.Init.NSS = SPI_NSS_HARD_INPUT;
    if (HAL_SPI_Init(&hspi1) != HAL_OK) Error_Handler();

    GPIO_InitTypeDef gpio = {0};
    gpio.Pin = SPI_NSS_RX_ece2_Pin;
    gpio.Mode = GPIO_MODE_AF_PP;
    gpio.Pull = GPIO_PULLUP;
    gpio.Speed = GPIO_SPEED_FREQ_LOW;
    gpio.Alternate = GPIO_AF5_SPI1;
    HAL_GPIO_Init(SPI_NSS_RX_ece2_GPIO_Port, &gpio);
}

void polling_routine(void)
{
    uint8_t rx = 0U;
    HAL_StatusTypeDef result = HAL_SPI_Receive(&hspi1, &rx, 1U, 2000U);
    if (result == HAL_OK)
        printf("TASK11 FPGA->MCU RX=%02X\r\n", (unsigned int)rx);
    else if (result == HAL_TIMEOUT)
        printf("TASK11 RX timeout: check FPGA SCK MOSI CS GND\r\n");
    else
        printf("TASK11 RX error=%d\r\n", (int)result);
}
```

그림 19. 과제11 FPGA→MCU 수신 코드

```
Executing task in folder FPGA LAB7: C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe -NoProfile -ExecutionPolicy Bypass -File C:\Users\young\Documents\fpga_lab7\ste32/tools/activate_and_build.ps1 -Task task11

Cleaning... 31 files.
ninja: Entering directory `C:\Users\young\Documents\ece2_474\build\Debug'
[31/31] Linking C executable ece2_474.elf
Memory region         Used Size  Region Size  %age Used
           RAM:        2640 B       128 KB      2.01%
         FLASH:       30648 B       512 KB      5.85%
ACTIVE task11; backup=C:\Users\young\Documents\ece2_474\lab7_backup\20261102-120352_before_task11
```

그림 20. 과제11 STM32 빌드 성공 결과

```
`timescale 1ns/1ps
module spi_master_tx #(
    parameter integer SCK_DIV=25,
    parameter integer GAP_CYCLES=50_000_000
)(
    input wire clk_50mhz, input wire rst_p,
    output reg spi_sck, output reg spi_cs_n, output reg spi_mosi,
    output reg [7:0] tx_data, output reg done
);
    reg [31:0] div_count=0, gap_count=0;
    reg [3:0] edge_count=0;
    reg busy=0;
    always @(posedge clk_50mhz) begin
        if(rst_p) begin spi_sck<=0;spi_cs_n<=1;spi_mosi<=0;tx_data<=8'h55;done<=0;div_count<=0;gap_count<=0;edge_count<=0;busy<=0; end
        else begin
            done<=0;
            if(!busy) begin
                spi_sck<=0;spi_cs_n<=1;div_count<=0;
                if(gap_count==GAP_CYCLES-1) begin gap_count<=0;busy<=1;spi_cs_n<=0;spi_mosi<=tx_data[7];edge_count<=0; end
                else gap_count<=gap_count+1;
            end else if(div_count==SCK_DIV-1) begin
                div_count<=0;
                if(!spi_sck) begin spi_sck<=1; edge_count<=edge_count+1; end
                else begin
                    spi_sck<=0;
                    if(edge_count==8) begin spi_cs_n<=1;busy<=0;done<=1;tx_data<={tx_data==8'h55}?8'hA5:8'h55; end
                    else spi_mosi<=tx_data[7-edge_count];
                end
            end else div_count<=div_count+1;
        end
    end
endmodule

module spi_master_top(input wire clk_50mhz,input wire rst_p,output wire spi_sck,output wire spi_cs_n,output wire spi_mosi);
    wire [7:0] tx_data; wire done;
    spi_master_tx u_tx(clk_50mhz,rst_p,spi_sck,spi_cs_n,spi_mosi,tx_data,done);
endmodule
```

그림 21. 과제11 FPGA SPI 마스터 송신 코드



그림 22. 과제11 SPI 마스터 자기검사 PASS 결과

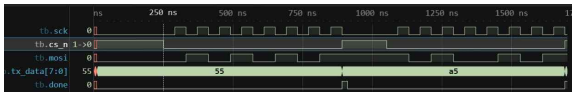


그림 23. 과제11 55·A5 송신 및 done 파형

MCU는 과제9의 SPI1 슬레이브 설정을 재사용한다. FPGA SCK·MOSI·CS를 각각 MCU PB3·PA7·PA4에 연결하고 VCP UART의 RX 값이 FPGA의 송신값과 같은지 확인한다.

8. 과제12 · FPGA→FPGA 송수신

8.1 통합 구성

FPGA Master	→	SCK·MOSI·CS	→	FPGA Slave	→	UART TX	→	PC

그림 6. FPGA 간 SPI 송수신 및 확인 경로

```
`timescale 1ns/1ps
module spi_master_core #(parameter integer SCK_DIV=25, GAP_CYCLES=50_000_000)(
    input wire clk,input wire rst,output reg sck,output reg cs_n,output reg mosi,output reg [7:0] tx_data,output reg done);
    reg [31:0] div_count=0,gap_count=0; reg [3:0] edges=0; reg busy=0;
    always @(posedge clk) begin
        if(rst) begin sck<=0;cs_n<=1;mosi<=0;tx_data<=8'h55;done<=0;div_count<=0;gap_count<=0;edges<=0;busy<=0; end
        else begin done<=0;
            if(!busy) begin sck<=0;cs_n<=1;div_count<=0;
                if(gap_count==GAP_CYCLES-1) begin gap_count<=0;busy<=1;cs_n<=0;mosi<=tx_data[7];edges<=0; end
                else gap_count<=gap_count+1;
            end else if(div_count==SCK_DIV-1) begin div_count<=0;
                if(!sck) begin sck<=1;edges<=edges+1; end
                else begin sck<=0;
                    if(edges==8) begin cs_n<=1;busy<=0;done<=1;tx_data<={tx_data==8'h55}?8'hA5:8'h55; end
                    else mosi<=tx_data[7-edges];
                end
            end else div_count<=div_count+1;
        end
    end
endmodule

module spi_slave_core(input wire clk,input wire rst,input wire sck_in,input wire cs_n_in,input wire mosi_in,
                      output reg [7:0] rx_data,output reg rx_valid);
    reg [1:0] ss=0,cs=3,ms=0; reg prev=0; reg [7:0] shift=0; reg [2:0] count=0;
    wire rise=ss[1]&~prev;
    always @(posedge clk) begin
        if(rst) begin ss<=0;cs<=3;ms<=0;prev<=0;shift<=0;count<=0;rx_data<=0;rx_valid<=0; end
        else begin ss<={ss[0],sck_in};cs<={cs[0],cs_n_in};ms<={ms[0],mosi_in};prev<=ss[1];rx_valid<=0;
            if(cs[1]) count<=0;
            else if(rise) begin shift<={shift[6:0],ms[1]};
                if(count==7) begin rx_data<={shift[6:0],ms[1]};rx_valid<=1;count<=0; end
                else count<=count+1;
            end
        end
    end
endmodule

module spi_fpga_link #(parameter integer SCK_DIV=25,GAP_CYCLES=50_000_000)(
    input wire clk_50mhz,input wire rst_p,output wire spi_sck,output wire spi_cs_n,output wire spi_mosi,
    output wire [7:0] tx_data,output wire [7:0] rx_data,output wire rx_valid,output wire done);
    spi_master_core #(.SCK_DIV(SCK_DIV),.GAP_CYCLES(GAP_CYCLES)) master(clk_50mhz,rst_p,spi_sck,spi_cs_n,spi_mosi,tx_data,done);
    spi_slave_core slave(clk_50mhz,rst_p,spi_sck,spi_cs_n,spi_mosi,rx_data,rx_valid);
endmodule
```

그림 24. 과제12 FPGA 마스터·슬레이브 통합 코드

```
Executing task in folder FPGA LAB7: C:/Users/young/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/python.exe C:\Users\young\Documents\fpga_lab7\circuits\tools/fpga_lab.py simulate

RUN C:\msys64\mingw64\bin\iverilog.EXE -g2012 -Wall -s tb -o C:\Users\young\Documents\fpga_lab7\circuits\spi_fpga_link\build\sim\sim.vvp C:\Users\young\Documents\fpga_lab7\circuits\spi_fpga_link\src\spi_fpga_link.v C:\Users\young\Documents\fpga_lab7\circuits\spi_fpga_link\sim\tb_design.v
VCD info: dumpfile wave.vcd opened for output.
PASS FPGA SPI LINK: master 55/A5 received by slave
C:\Users\young\Documents\fpga_lab7\circuits\spi_fpga_link\sim\tb_design.v:111: $finish called at 3130000 (1ps)
WAVE C:\Users\young\Documents\fpga_lab7\circuits\spi_fpga_link\build\sim\wave.vcd
Terminal will be reused by tasks, press any key to close it.
```

그림 25. 과제12 FPGA 통합 링크 자기검사 PASS 결과

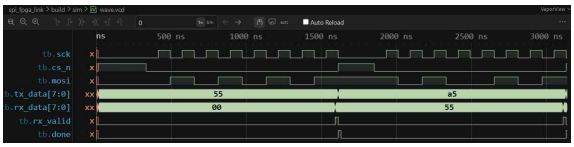


그림 26. 과제12 FPGA 통합 링크 송수신 파형

과제11의 마스터 송신 회로와 과제10의 슬레이브 수신 회로를 지정 조의 FPGA 두 대에 각각 적용한다. 수신 FPGA는 받은 바이트를 9600 baud UART로 PC에 출력한다. 한 방향 검증 후 두 보드의 역할을 교대한다.

8.2 성공 판정

검증 항목	성공 조건
비트 순서	송신값과 수신값 완전 일치
프레임 경계	CS별 정확히 8 SCK 상승 에지
반복 동작	1초 간격 바이트가 누락 없이 출력
역할 교대	양쪽 보드 모두 TX·RX 성공
리셋	CS=1, SCK=0으로 안전 복귀

표 8. 과제12 성공 판정 기준

8.3 예상 문제와 대응

- 수신값 비트 이동: Mode 0 에지와 첫 MSB 준비 시점 재확인

- 항상 FF/00 수신: MOSI·GND·CS 연결 및 핀 방향 확인
- 간헐적 오류: 비동기 입력 동기화, SCK 주파수와 펄스 폭 확인
- 수신 없음: CS 극성, FPGA XDC, MCU PA4 NSS 설정 확인

9. 본 실험 진행 계획

순서	예정 작업	확인할 내용
1	MCU 송신 빌드	TX=55, result=0
2	오실로스코프 측정	CS·SCK·MOSI, 주파수
3	MCU→MCU	RX 값·역할 교대
4	FPGA RX TB	4개 패턴·CS 재시작
5	MCU→FPGA	FPGA UART 수신값
6	FPGA TX TB	Mode 0·SCK≤1 MHz
7	FPGA→MCU	VCP UART 수신값
8	FPGA→FPGA	수신 UART·역할 교대
9	증빙 확보	파형·콘솔·사진·영상

표 9. LAB7 본 실험 예정 절차

9.1 실험 전 체크리스트

- STM32_3 송신·수신 코드와 9600 VCP UART 설정
- FPGA SPI slave/master RTL 및 자기검사 테스트벤치
- FPGA 프로젝트의 SCK·MOSI·CS·UART XDC
- 점퍼선 4개 이상, 공통 GND, 오실로스코프 프로브
- 송신 패턴 0x55·0xA5·0x00·0xFF 기록표

실험 후에는 실제 측정 주파수, 송수신값, 예상과 실제 결과의 차이, 문제 해결 과정 및 최종 태그·커밋 해시를 결과보고서에 기록한다.

10. 자료 링크 및 참고문헌

10.1 실습 자료

- STM32 실습 저장소: https://github.com/Glaysia/ece2_474
- 수업 자료 저장소: https://github.com/Glaysia/ece2_26_2_2
- STM32_3 매뉴얼: ece2_474/docs/STM32_3.md
- 실습 버전: v1.1.0
- 개인 LAB7 저장소·태그·커밋: 실험 자료 업로드 후 확정하여 결과보고서에 기재

10.2 참고문헌

[1] 이해리, ‘STM32 실습 안내’, STM32_4week_plan, 2026, pp. 6, 11–12.

[2] Glaysia, ece2_474/docs/STM32_3.md, ‘SPI로 MCU와 FPGA 연결’, v1.1.0, 2026.

[3] STMicroelectronics, RM0440 Reference Manual: STM32G4 Series, SPI chapter.

[4] STMicroelectronics, UM2505, Getting started with STM32G4 Nucleo-64 board.

[5] ARM, Cortex-M4 Devices Generic User Guide.

10.3 출처 대응

SPI 통신 조건과 과제 범위는 [1]·[2], MCU 주변장치와 보드 핀은 [3]·[4], CPU와 예외 처리의 기본 구조는 [5]를 참고하였다. 실제 구현 결과와 개인 저장소의 고정 링크는 본 실험 이후 결과보고서에 추가한다.

10.4 개인 저장소 및 고정 증빙 링크

아래 주소는 예비보고서에 사용한 소스·캡처·시뮬레이션 자료를 기준 커밋과 태그에 고정한 주소이다. main 브랜치가 변경되어도 동일한 증빙을 확인할 수 있다.

예비보고서 링크:

https://github.com/dalkomys777/fpga_lab7/blob/main/reports/pre/%EC%98%88%EB%B9%84%EB%A6%AC%ED%8F%AC%ED%8A%B8_LAB7%202025440031%EA%B9%80%EC%98%81%EC%84%9C.pdf

- 저장소 URL: https://github.com/dalkomys777/fpga_lab7
- 기준 브랜치: main
- 증빙 기준 태그: prelab-evidence-v1.0.0
- 증빙 기준 커밋: 2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c
- 증빙 커밋 주소: https://github.com/dalkomys777/fpga_lab7/commit/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c

• 기준 태그 주소:
https://github.com/dalkomys777/fpga_lab7/tree/prelab-evidence-v1.0.0

• 사용자 캡처 폴더:
https://github.com/dalkomys777/fpga_lab7/tree/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/evidence/user_captures

• 예비 검증 자료 폴더:
https://github.com/dalkomys777/fpga_lab7/tree/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/evidence/prelab

• 증빙 설명:
https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/evidence/README.md

• 프로젝트 설명:
https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/README.md

• VS Code 작업공간:
https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/LAB7.code-workspace

10.5 STM32 소스 코드

• 과제8 SPI 파형 확인:
https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/stm32/task8_spi_scope/ece2_474_routine.c

• 과제9 MCU 마스터:
https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/stm32/task9_mcu_to_mcu/master/ece2_474_routine.c

• 과제9 MCU 슬레이브:
https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/stm32/task9_mcu_to_mcu/slave/ece2_474_routine.c

• 과제10 MCU→FPGA:
https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/stm32/task10_mcu_to_fpga/ece2_474_routine.c

• 과제11 FPGA→MCU:
https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/stm32/task11_fpga_to_mcu/ece2_474_routine.c

• 과제12 안내:
https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/stm32/task12_fpga_to_fpga/README.md

10.6 FPGA 소스·테스트벤치·원본 파형

• SPI 슬레이브 RTL:
https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/circuits/spi_slave/src/spi

_slave.v

- SPI 슬레이브 테스트벤치: https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/circuits/spi_slave/sim/tb_design.v
- SPI 슬레이브 VCD: https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/evidence/prelab/spi_slave/wave.vcd
- SPI 마스터 RTL: https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/circuits/spi_master/src/spi_master.v
- SPI 마스터 테스트벤치: https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/circuits/spi_master/sim/tb_design.v
- SPI 마스터 VCD: https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/evidence/prelab/spi_master/wave.vcd
- FPGA 통합 링크 RTL: https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/circuits/spi_fpga_link/src/spi_fpga_link.v
- FPGA 통합 링크 테스트벤치: https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/circuits/spi_fpga_link/sim/tb_design.v
- FPGA 통합 링크 VCD: https://github.com/dalkomys777/fpga_lab7/blob/2335d2f4830b73e47bc9b7a5e4eaff70f2e3881c/evidence/prelab/spi_fpga_link/wave.vcd
- 재현 방법: git checkout prelab-evidence-v1.0.0 후 LAB7.code-workspace를 열고 SPI Slave, SPI Master, FPGA SPI Link 시뮬레이션 작업을 실행한다.